

C++ Notes

Copy vs Move semantics & Universal References

Adam HAMOUCH

May 4, 2026

1. Définition

En C++, on oppose généralement la **copy semantics** à la **move semantics** :

- **Copy** : duplication d'une ressource (coûteux)
- **Move** : transfert de ressource (rapide)

Les **universal references** (écrites **T&&** dans un template) permettent d'adapter automatiquement le comportement :

- si on passe une **lvalue** → copy
- si on passe une **rvalue** → move

2. Problème résolu

- Éviter les copies inutiles (performance)
- Écrire une seule fonction capable de gérer copy **et** move
- Permettre un code générique efficace (templates)

3. Fonctionnement

- `std::move` : force un move (cast en rvalue)
- `std::forward` : conserve la nature originale (lvalue ou rvalue)

Règle clé :

- rvalue reference → utiliser `std::move`
- universal reference → utiliser `std::forward`

4. Exemple minimal

```
#include <vector>
#include <utility>

struct Component {};

class Entity {
public:
    template<typename T>
    void AddComponent(T&& component)
    {
        components.emplace_back(std::forward<T>(component));
    }

private:
    std::vector<Component> components;
};
```

5. Pièges classiques

- Oublier `std::forward` → toujours copy
- Utiliser `std::move` sur une universal reference → casse les lvalues
- Forward plusieurs fois → double move potentiel
- Utiliser une variable après un move → état indéfini
- Faire `std::move` sur une variable retournée → casse la RVO

6. Coût et performance

- Copy : coûteux (allocation, duplication mémoire)
- Move : rapide (transfert de pointeurs / ressources)
- Forward : aucun coût réel (cast à la compilation)

Très important en moteur de jeu :

- éviter copies de mesh / textures / buffers
- privilégier move ou construction directe (`emplace`)

7. Quand utiliser / éviter

7.1 Utiliser

- Templates génériques

- APIs type ECS / containers
- Code performance critique

7.2 Éviter

- Code simple non générique
- Quand la lisibilité prime
- Si on ne maîtrise pas bien move / forward